

Министерство просвещения Российской Федерации
[Полное наименование органа управления образованием]
Муниципальное бюджетное общеобразовательное учреждение
«[Полное наименование школы]»



ИНДИВИДУАЛЬНЫЙ ИТОГОВЫЙ ПРОЕКТ

Разработка веб-приложения «English Today and Tomorrow»
для изучения английской лексики
методом визуальных ассоциаций

Направление: Информатика и ИКТ

Автор проекта: [Фамилия Имя Отчество],
ученик 9 класса

Руководитель: [Фамилия И. О.],
учитель информатики

Допущен к защите

Протокол педагогического совета № [___] от [__] мая 2026 г.

[Город] — 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЕКТА	5
1.1. Три принципа памяти: двойное кодирование, активное вспоминание, интервальное повторение	5
1.2. Обзор аналогов и свободная ниша	6
1.3. Почему HTML/CSS/JavaScript и отказ от сервера и конструкторов	7
1.4. REST API Unsplash и метод «vibe coding» с ИИ-ассистентом	8
1.5. LocalStorage: 5 МБ, JSON.stringify/parse и отказ от Base64	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ	10
2.1. Функциональные и нефункциональные требования	10
2.2. Архитектура «четыре файла и одно хранилище»	11
2.3. Структура базы: поля папки и слова, связь folderId	12
2.4. Ключевые алгоритмы: групповой рендер, карточка-перевёртыш, режим повторения, парсер импорта	13
2.5. Интерфейс: закреплённая панель, липкие заголовки, палитра, адаптивность	15
2.6. Брендинг: имя, слоган, логотип	16
ГЛАВА 3. ТЕСТИРОВАНИЕ, ОШИБКИ И ОЦЕНКА КАЧЕСТВА	17
3.1. Методика функционального тестирования в Chrome и на телефоне	17

3.2.	Четыре ошибки, которые меня научили	18
3.3.	Инструкция пользователя	19
3.4.	Опрос 15 одноклассников	20
ЗАКЛЮЧЕНИЕ		21
СПИСОК ИСТОЧНИКОВ		22
ПРИЛОЖЕНИЕ А. Подписи рисунков		23
ПРИЛОЖЕНИЕ Б. Фрагменты кода		24
ПРИЛОЖЕНИЕ В. Результаты опроса		25

ВВЕДЕНИЕ

Всё началось с обычного списка слов. В сентябре учитель английского языка велел нам к понедельнику выучить сорок новых лексических единиц по теме «Путешествия». Я завёл в тетради аккуратную табличку: слева — английское слово, справа — русский перевод. На следующий день повторил, через два дня — половина слов забылась, через неделю в памяти осталась едва ли треть. Обычная школьная история, и оттого особенно обидная: я честно потратил время, а результат не оправдал ожиданий.

Я стал думать, почему так происходит. Оказалось, что мой бумажный список нарушает сразу три принципа, о которых пишет любая книга по психологии памяти. Во-первых, слово существует только в текстовой форме — нет зрительного образа. Во-вторых, повторение пассивное: я просто читаю пару «английское — русское», и мозг не напрягается. В-третьих, нет интервалов: я повторяю всё подряд, не отличая то, что уже знаю, от того, что ещё не отложилось. Так родилась идея приложения, которое само подсказывает картинку, само подсовывает слово для активного вспоминания и само группирует слова так, чтобы повторение было осмысленным.

Я назвал приложение **«English Today and Tomorrow»** (ETT), а слоганом выбрал фразу «Учи сегодня — говори завтра». Это бесплатно, работает прямо в браузере и не требует ни регистрации, ни установки. Все слова хранятся в памяти самого браузера (LocalStorage), поэтому приложение продолжает работать даже без интернета — кроме поиска картинок, для которого используется API Unsplash.

Цель проекта — разработать бесплатное веб-приложение для изучения английской лексики методом визуальных ассоциаций, которое объединит тематические папки-группы, карточки-перевёртыши, режим самопроверки и импорт реальных словарей, и будет работать полностью в браузере без серверной части.

Для достижения цели поставлены **шесть задач**:

1. Изучить психологические основы запоминания лексики и провести обзор существующих аналогов.
2. Спроектировать архитектуру приложения и выбрать технологии разработки.
3. Реализовать встроенный словарь из 144 слов (12 системных папок × 12 единиц) с примерами и изображениями.
4. Реализовать карточки-перевёртыши, режим повторения и живой поиск.
5. Реализовать импорт реальных словарей (SMARTool A1–B2, простой CSV/TXT/JSON) с пятью режимами раскладки по группам.
6. Провести функциональное тестирование, исправить ошибки и опросить одноклассников.

Объект исследования — процесс самостоятельного изучения английской лексики школьником. **Предмет** исследования — веб-приложение как инструмент организации этого процесса методом визуальных ассоциаций.

Методы, которые я использовал: анализ научной литературы по психологии памяти, сравнительный анализ существующих приложений-аналогов, метод прототипирования («vibe coding» совместно с ИИ-ассистентом), функциональное тестирование в браузере Chrome и на смартфоне, анкетный опрос 15 одноклассников.

Практическая значимость работы в том, что приложение полностью готово к использованию: оно бесплатное, работает офлайн (кроме поиска картинок), не требует регистрации, и любой ученик может либо открыть его в браузере, либо импортировать собственный словарь и сразу начать заниматься. Тот же подход пригодится и для подготовки к ОГЭ по английскому языку, где лексический запас играет ключевую роль.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОЕКТА

1.1. Три принципа памяти: двойное кодирование, активное вспоминание, интервальное повторение

Прежде чем писать код, я постарался понять, как вообще работает память — иначе получилось бы очередное приложение, которое красиво выглядит, но ничему не учит. Оказалось, что у учёных есть понятные и проверенные принципы, и на них можно опереться. Этих принципов три, и все три легли в основу приложения ЕТТ.

Первый принцип — двойное кодирование (Allan Paivio, 1971). Канадский психолог Аллан Пайвио в своей теории двойного кодирования утверждает, что человек запоминает информацию лучше, если она одновременно поступает через два независимых канала — вербальный и зрительный. Слово «apple», поданное только текстом, кодируется одним каналом. Слово «apple» рядом с фотографией красного яблока кодируется сразу двумя, и вероятность долговременного удержания удваивается. Именно поэтому в ЕТТ у каждого слова есть imageUrl — прямая ссылка на изображение с Unsplash, которая подгружается и в режиме повторения, и при просмотре карточки.

Второй принцип — активное вспоминание. В когнитивной психологии считается, что пассивное чтение переводов практически не тренирует память, тогда как попытка вспомнить ответ до того, как он показан, укрепляет нейронную связь в несколько раз. На этом работает любой тест с закрытой книгой. Поэтому карточка в ЕТТ спроектирована как перевёртыш: на лицевой стороне — русское слово и подсказка «нажми — по-английски», а на обороте — английское слово и пример. Мозг вынужден сперва попытаться достать перевод из памяти, и лишь потом пользователь сам переворачивает карточку, проверяя себя.

Третий принцип — интервальное повторение и связанная с ним система Лейтнера. Немецкий журналист Себастьян Лейтнер в 1970-х годах предложил метод коробок: новые карточки попадают в первую коробку и повторяются часто, а те, которые ученик вспомнил, переходят во вторую, потом в третью и так далее — интервалы растут. В

ЕТТ я реализую упрощённую версию: режим повторения по группе, где пользователь сам отмечает «Знаю» или «Не знаю», и в конце получает статистику. В перспективе именно эти метки должны стать основой для автоматического планировщика по Лейтнеру — об этом написано в заключении.

1.2. Обзор аналогов: Quizlet, Duolingo, Lingualeo, Anki и свободная ниша

Прежде чем начать разработку, я изучил четыре популярных приложения для изучения лексики. Каждое из них по-своему сильное, но у каждого есть ограничение, которое мешает школьнику начать пользоваться им прямо сейчас, без регистраций и подписок.

Quizlet — самый известный сервис карточек. У него отличный режим «тренировки» и красивые анимации, но с 2022 года почти все полезные функции (например, режим «пишите» и расширенная статистика) закрыты платной подпиской Quizlet Plus. Кроме того, сервис требует обязательной регистрации даже для просмотра чужих наборов. Для школьника, у которого нет своей почты или который не хочет её оставлять, это серьёзный барьер.

Duolingo — это не приложение для лексики, а полноценный курс с грамматикой и аудированием. Его gamification (очки, ударный режим, лиги) затягивает, но конкретно лексику по школьной теме он не повторяет: тематические наборы фиксированы, нельзя импортировать свой список слов. Подходит для общего развития, но не для подготовки к контрольной.

Lingualeo — российский сервис с хорошим словарём и тренировками, но он тоже требует регистрации, навязывает платный Jungle-курс и заметно подтормаживает в браузере из-за тяжёлой верстки. Бесплатная версия ограничена по числу слов в день.

Anki — мощная программа с интервальным повторением, но её интерфейс выглядит как из 2007 года, настройка колод требует терпения, а мобильное приложение стоит денег. Технически Anki идеален, практически — сложен для школьника.

Вывод о свободной нише. Ниша, в которую я пытаюсь попасть, формулируется так: бесплатное веб-приложение, которое (а) открывается по ссылке без регистрации; (б) работает офлайн за счёт `LocalStorage`; (в) позволяет импортировать реальные словари; (г) имеет красивый современный интерфейс. Ни один из перечисленных аналогов не закрывает все четыре пункта одновременно. ЕТТ закрывает.

1.3. Почему HTML/CSS/JavaScript и отказ от сервера и конструкторов

Технологию для проекта я выбирал между тремя вариантами: конструктор сайтов (Tilda, Wix), фреймворк с бэкендом (Next.js или Django) и чистый стек HTML5 + CSS3 + Vanilla JavaScript. Конструктор отпал сразу: он не позволяет нормально работать с `LocalStorage`, не даёт свободы в стилизации карточек-перевёртышей и, главное, не учит программировать. Фреймворк с бэкендом дал бы больше возможностей, но потребовал бы сервера, базы данных и регистрации пользователей — а вся суть проекта в том, чтобы он был бесплатным и работал офлайн. Остаётся третий путь.

Стек HTML5 + CSS3 + Vanilla JavaScript (то есть без библиотек) выбран потому, что это самый «честный» стек для школьного проекта: он не прячет за абстракциями то, как работает веб-страница. Любое событие — клик, ввод текста, поиск — обрабатывается через `addEventListener`, любое изменение интерфейса — через прямую работу с DOM. Это позволяет понять базовые механизмы браузера, что ценнее, чем умение вызывать готовый метод из библиотеки.

Важный плюс — отсутствие бэкенда. Не нужен ни сервер, ни база данных, ни хостинг. Все данные пользователя лежат в его собственном браузере в `LocalStorage`, и никто, кроме него, к ним доступа не имеет. Это снимает вопросы конфиденциальности и GDPR: я как разработчик физически не могу слить данные пользователя, потому что они у него на устройстве. Единственный внешний запрос — к Unsplash API за картинками, и тот можно заменить прямой ссылкой на готовое фото.

Ещё один плюс — `_portability`. Проект состоит из четырёх файлов (`index.html`, `style.css`, `script.js`, `data.js`), которые можно открыть двойным кликом в любом браузере без установки чего-либо. На уроке информатики я могу скинуть эти файлы через флешку или мессенджер, и у одноклассника мгновенно заработает точно такое же приложение со своим словарём. Сервер так не умеет.

1.4. REST API Unsplash и метод разработки «vibe coding» с ИИ-ассистентом

Для картинок я использую Unsplash — бесплатную фотобанку с открытым REST API. Это значит, что мое приложение отправляет HTTP-запрос на адрес `https://api.unsplash.com/search/photos` с параметрами `query` (что искать) и `per_page=3`, получает обратно JSON и показывает пользователю три превью. Запрос идёт прямо из браузера через `fetch()`, ответ парсится через `response.json()`. Для авторизации используется Access Key, который пользователь подставляет в константу `UNSPLASH_KEY` вместо плейсхолдера `'YOUR_UNSPASH_ACCESS_KEY'`. Этот ключ регистрируется бесплатно на сайте `unsplash.com/developers` за пять минут.

REST API — это архитектурный стиль, при котором сервер предоставляет ресурс по HTTP, а клиент общается с ним методами GET, POST и так далее. В моём случае достаточно GET-запроса: я только читаю публичные фото, ничего не отправляю и не модифицирую. У Unsplash есть ограничение — 50 запросов в час на бесплатном тарифе, но для школьного проекта этого хватает с большим запасом. Если лимит исчерпан, в коде предусмотрена заглушка: вместо реального поиска подставляются три случайные картинки с `source.unsplash.com`.

Сама разработка велась методом, который в индустрии называют «vibe coding» — это когда человек пишет спецификацию и держит в голове цель, а рутинный код помогает генерировать ИИ-ассистент. Я заранее составил файл `SPEC.md`, в котором по пунктам описал, какие функции, какие интерфейсы и какие данные должны быть в проекте. Затем в диалоге с ИИ-ассистентом я просил реализовать отдельные части спецификации, проверял результат, возвращался к

SPEC и правил. Подробно об этом подходе и о том, как однажды потеря контекста привела к появлению файла SPEC.md, рассказано в главе 3.2.

«Vibe coding» не означает, что ИИ пишет за меня всю программу. Скорее, это ускорение: рутинные конструкции (рендеринг списка, обработка клика, парсинг CSV) ассистент генерирует за секунды, а я трачу время на смысловые решения — как должны вести себя липкие заголовки, что должно происходить при пустом поиске, как корректно разобрать кавычки в CSV. Важно, что финальный код я проверяю и понимаю, потому что на защите проекта придётся объяснять каждую строку.

1.5. LocalStorage: 5 МБ, JSON.stringify/parse и отказ от Base64

LocalStorage — это встроенное в браузер хранилище «ключ-значение». У каждого сайта есть свой LocalStorage, доступный через JavaScript: можно записать значение по ключу (`localStorage.setItem('ett_words', '[...])`), прочитать (`localStorage.getItem('ett_words')`) и удалить. Объём хранилища ограничен примерно 5 МБ на домен — этого с запасом хватает для тысяч карточек, но **не** хватает, если пытаться туда же записывать картинки в формате Base64. Об этом — ниже.

Данные в LocalStorage хранятся как строки. Поэтому массив объектов с папками и словами я превращаю в строку через `JSON.stringify(folders)`, а при чтении — обратно через `JSON.parse(localStorage.getItem('ett_folders'))`. Эти функции — часть стандартного JavaScript, ничего подключать не нужно. Если при чтении возникает ошибка (например, данные повреждены), я в блоке `try/catch` откатываюсь к встроенному словарю из `data.js` и перезаписываю LocalStorage заново. Это гарантирует, что приложение всегда запускается, даже если пользователь случайно почистил часть данных.

Самая поучительная история с LocalStorage — попытка хранить там картинки в формате Base64. Base64 — это способ закодировать бинарный файл в строку текста. Я думал: «А что, если дать пользова-

телю загружать свои фото?» и пробовал через `FileReader.readAsDataURL` превращать выбранную картинку в Base64-строку и класть её в `imageUrl`. Оказалось, что одна фотография среднего размера в Base64 занимает около 200 КБ. Десяток таких фото — и лимит LocalStorage в 5 МБ исчерпан. Браузер молча выбрасывает исключение `QuotaExceededError`, и все новые данные теряются. Я откатил это решение и теперь храню только прямые ссылки на Unsplash — это пара сотен байт на слово. Подробности — в главе 3.2, «Четыре ошибки».

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

2.1. Функциональные и нефункциональные требования

Прежде чем написать первую строку кода, я составил список требований к приложению. Требования делятся на две группы: функциональные (что приложение должно делать) и нефункциональные (каким требованиям оно должно соответствовать, помимо функций).

Функциональные требования:

- просмотр слов, сгруппированных по 12 системным тематическим папкам (Еда 🍎, Животные 🐾, Семья 👨‍👩‍👧, Школа 🏫, Погода 🌤️, Спорт ⚽, Путешествия ✈️, Эмоции 😊, Дом 🏠, Одежда 👕, Природа 🌳, Технологии 💻);
- создание пользовательских папок и удаление их с подтверждением;
- добавление, редактирование, удаление, перемещение и копирование слов;
- переворот карточки по клику (CSS 3D-эффект `rotateY`);
- живой поиск по слову и переводу;
- режим повторения с переключением направления EN → RU / RU → EN и статистикой;
- импорт словарей SMARTool и простых CSV/TXT/JSON с пятью режимами раскладки по группам;

- поиск картинок через Unsplash API и прямую ссылку URL.

Нефункциональные требования:

- работа в браузерах Chrome, Firefox, Safari без установки чего-либо;
- адаптивность под десктоп, планшет и телефон (брейкпоинты 860px и 640px);
- отсутствие бэкенда, библиотек и фреймворков;
- продолжение работы офлайн (LocalStorage), кроме поиска картинок;
- время отклика интерфейса — не более 100 мс на рендер 100 карточек;
- единообразие визуального языка (палитра, шрифт, отступы) через CSS-переменные.

Этот список лёг в основу файла `SPEC.md`, который я держал перед глазами на протяжении всей разработки. Каждое требование можно проверить — либо оно выполнено, либо нет, промежуточных состояний нет.

2.2. Архитектура «четыре файла и одно хранилище»

Архитектура приложения максимально проста, и в этой простоте — её сила. Проект состоит ровно из четырёх файлов: `index.html` описывает структуру страницы (верхняя панель, лента вкладок, основная область, модальные окна, оверлей повторения); `style.css` содержит все стили, организованные через CSS-переменные в `:root`; `script.js` хранит всю логику и сгруппирован по функциям с осмысленными именами; `data.js` содержит встроенный словарь из 144 слов и 12 системных папок.

Пятое хранилище — это LocalStorage браузера, в который при первом запуске копируются данные из `data.js`. С этого момента все изменения, которые делает пользователь (новые слова, новые папки, импортированные словари), сохраняются только в LocalStorage. Встроенный словарь в `data.js` остаётся как «нулевая точка» — на случай, если пользователь захочет сбросить всё к заводскому состо-

янию (тогда можно очистить LocalStorage, и при следующем запуске словарь скопируется заново).

Поток данных выглядит так: при загрузке страницы функция `loadState()` читает из LocalStorage ключи `ett_folders` и `ett_words`. Если они пустые — копирует туда системный словарь из `window.SYSTEM_FOLDERS` и `window.SYSTEM_WORDS`. Затем `renderAll()` вызывает `renderTabs()` (лента вкладок-папок) и `renderGroups()` (основная область с секциями). Любое действие пользователя (клик по карточке, ввод в поиск, добавление слова) изменяет состояние в памяти и тут же вызывает `saveState()` + `renderAll()`. Так интерфейс всегда отражает актуальное состояние, а данные не теряются между сессиями.

2.3. Структура базы: поля папки и слова, связь `folderId`

Хотя в проекте нет настоящей реляционной базы данных, данные спроектированы реляционно: две сущности (папка и слово) связаны внешним ключом. Это упрощает будущий перенос на сервер, если такой шаг когда-нибудь понадобится.

Папка (folder) имеет три поля: `id` — уникальный идентификатор вида `f1`, `f2` ... `f12` для системных и `f<временная_метка>` для пользовательских; `name` — название («Еда», «Животные», «A1 · Еда»); `icon` — эмодзи-иконка (, , , , ); `isSystem` — булев флаг, равный `true` для 12 системных папок и `false` для пользовательских. Системные папки нельзя удалить через интерфейс — это гарантирует, что встроенный словарь всегда будет доступен.

Слово (word) имеет шесть полей: `id` — уникальный идентификатор вида `w101` ... `wc12`; `word` — английское слово (например, `apple`); `translation` — русский перевод (яблоко); `example` — короткое предложение-пример на английском (`I eat a green apple every morning.`); `folderId` — внешний ключ, ссылающийся на `id` папки; `imageUrl` — прямая ссылка на изображение с Unsplash в формате `https://images.unsplash.com/photo-...?auto=format&fit=crop&w=600&q=60`. У всех 144 системных слов поле `imageUrl` заполнено реальными проверенными ссылками.

Связь между сущностями — классическая «один-ко-многим»: одна папка содержит много слов, каждое слово принадлежит ровно одной папке. Когда пользователь копирует слово через кнопку , создаётся дубликат с теми же полями, но новым `id` и тем же `folderId`. Когда пользователь перемещает слово в другую папку, меняется только поле `folderId`. Когда пользователь удаляет пользовательскую папку, слова из неё не пропадают, а переносятся в первую системную папку (Еда) — это безопаснее, чем удалять весь словарь.

2.4. Ключевые алгоритмы: групповой рендер, карточка-перевёртыш, режим повторения, парсер импорта

Самый сложный алгоритм в проекте — это **групповой рендер секций**. Функция `renderGroups()` проходит по всем папкам, для каждой вызывает `sectionWords(folderId)`, которая фильтрует слова по `folderId` и по поисковому запросу, сортирует по английскому слову через `localeCompare`. Если в режиме поиска секция оказывается пустой, она не выводится совсем. Если поиск ничего не дал, выводится «пустое состояние» с двумя кнопками — «Добавить слово» и «Импорт словаря». Заголовок каждой секции сделан липким (`position: sticky`), чтобы при прокрутке длинного списка пользователь всегда видел, в какой он папке.

Второй алгоритм — **карточка-перевёртыш на чистом CSS 3D**. Каждая карточка состоит из контейнера `.card` с `perspective: 1000px`, внутри — `.card-inner` с `transform-style: preserve-3d` и `transition: transform .5s`. Две грани `.card-face` с `backface-visibility: hidden` расположены друг за другом: лицевая смотрит на пользователя (русское слово), обратная повёрнута на 180 градусов (`transform: rotateY(180deg)`) и содержит английское слово и пример. По клику на карточку JavaScript добавляет класс `is-flipped`, который применяет к `.card-inner` поворот `rotateY(180deg)` — и пользователь видит обратную сторону. Никаких библиотек, никаких анимаций JavaScript — только CSS-движок браузера.

Третий алгоритм — **режим повторения**. Функция `openStudy(folderId)` собирает список слов (всех или только выбранной папки), перемешивает его (`sort(() => Math.random() - 0.5)`) и открывает

полноэкранный оверлей. `renderFlipCard()` рисует большую flip-карточку с картинкой и словом. Кнопка `toggleDirection` переключает режим EN → RU / RU → EN. Кнопки «Знаю» и «Не знаю» увеличивают соответствующие счётчики и активируют кнопку «Дальше». Когда список заканчивается, `finishStudy()` показывает экран «Тема пройдена! 🎉» со статистикой — сколько слов, сколько «знаю», сколько «не знаю».

Четвёртый и самый объёмный алгоритм — **парсер импорта**. Функция `importWords(rawText, mode)` принимает текст словаря и режим раскладки (`level_topic` , `level` , `topic` , `alpha` , `one`). Сначала она пытается разобрать текст как JSON (`JSON.parse`). Если не получилось — как CSV, для чего используется корректный `parseCSVLine()` , который уважает кавычки и экранирование. Если в первой строке CSV найдены заголовки колонок SMARTool (`Target language lemma` , `User language gloss` , `Target language example sentence` , `Level` , `Topic(s)`), парсер понимает структуру и раскладывает слова по нужным полям. Если заголовков нет — текст разбирается как простой формат «слово, перевод[, пример]», где разделителем может быть запятая, табуляция, дефис с пробелами или точка с запятой.

После разбора для каждого слова определяется имя целевой папки по режиму: при `level_topic` — это «A1 · Еда»; при `level` — «A1», «A2», «B1», «B2» (с эмодзи 🎓); при `topic` — «Еда», «Животные» (с эмодзи 📖); при `alpha` — буква алфавита (с эмодзи abc); при `one` — единственная папка «Импортированные». Функция `findOrCreateFolder(name, mode)` ищет папку с таким именем, а если её нет — создаёт новую. Дубликаты внутри папки пропускаются: если слово с тем же английским написанием уже есть в целевой папке, новое не добавляется. В конце пользователь видит статус «✅ Импортировано слов: N. Создано папок: M.».

2.5. Интерфейс: закреплённая панель, липкие заголовки, палитра, адаптивность

Верхняя панель приложения всегда видна при прокрутке благодаря `position: fixed`. В ней слева — логотип и название, в центре — поле живого поиска, справа — четыре кнопки: «🎓 Повторение», «+»

Слово», «📁 Импорт», «ℹ️ О проекте». Под верхней панелью внутри той же шапки расположена липкая горизонтальная лента вкладок-папок: первая вкладка «📖 Все слова» со счётчиком, затем вкладки папок (эмодзи, имя, счётчик, у пользовательских — крестик ✕ для удаления), в конце — кнопка-вкладка «+ Папка» (создание через `prompt`).

В основной области слова выводятся группами: отдельная на каждую папку. Внутри секции — липкий заголовок (эмодзи, название, счётчик, кнопка «🎓 Повторить группу») и сетка карточек. Сетка адаптивная: `grid-template-columns: repeat(auto-fill, minmax(180px, 1fr))` автоматически подбирает число колонок в зависимости от ширины экрана.

Палитра задана CSS-переменными в `:root` и строго соответствует брендбуку: `--c-dark: #0f172a` для шапки, вкладок, футера и оборота карточки; `--c-bg: #f1f5f9` для фона страницы; `--c-text: #1e293b` для основного текста; `--c-accent: #38bdf8` для акцентов, ссылок и английского слова на обороте; `--c-white: #ffffff` для карточек. Радиус карточки — `12px` (`--r-card`), тень — `0 4px 14px rgba(15,23,42,.08)` (`--shadow-card`).

Адаптивность обеспечивают два медиа-запроса. На ширине `860px` (планшеты) строка поиска переходит на вторую строку панели, скрывается слоган. На ширине `640px` (телефоны) скрывается название рядом с логотипом, сетка карточек сжимается до `120px` минимальной ширины, уменьшаются шрифты. Это позволяет пользоваться приложением даже на небольшом смартфоне — что и было проверено в ходе тестирования (см. главу 3.1).

2.6. Брендинг: имя, слоган, логотип

Название «**English Today and Tomorrow**» (сокращённо ЕТТ) я выбрал потому, что оно отражает сразу две идеи. «Today» — это то, что происходит сейчас: пользователь открывает приложение, видит карточку, переворачивает её, запоминает. «Tomorrow» — это завтрашний день, когда это слово понадобится ему в разговоре. Имя рифму-

ется, легко запоминается и удобно сокращается до трёх букв, которые помещаются на логотип.

Слоган **«Учи сегодня — говори завтра»** усиливает эту идею и переводит её на русский язык. Он короткий, ритмичный и работает как обещание: если будешь учить сегодня, то завтра заговоришь. Это мотивирующее обещание — не угроза и не реклама, а мягкое напоминание о цели.

Логотип — квадрат со стороной 44px цвета `#0f172a`, рамкой толщиной 2px цвета `#38bdf8` и скруглением 10px. Внутри крупными белыми буквами — «ЕТТ», а рядом эмодзи книги 📖. Книга символизирует учебник, а акцентная рамка делает логотип заметным даже на тёмном фоне шапки. На мобильных размер уменьшается, но пропорции сохраняются. Эмодзи 📖 добавляет «человечности» и подсказывает, что это приложение именно про учебу.

Все три элемента брендинга работают вместе: имя задаёт направление, слоган обещает результат, логотип даёт опознавательный знак. Это базовый, но осознанный брендинг, без маркетингового бюджета — просто последовательность визуального языка во всех точках контакта пользователя с приложением.

ГЛАВА 3. ТЕСТИРОВАНИЕ, ОШИБКИ И ОЦЕНКА КАЧЕСТВА

3.1. Методика функционального тестирования в Chrome и на телефоне

Для тестирования я использовал браузер Google Chrome на ноутбуке (версия 124 для Windows) и смартфон Xiaomi Redmi Note 11 с Chrome для Android. Тестирование проводилось вручную по чек-листу, который я заранее выписал из SPEC.md. Каждый пункт чек-листа — это конкретное действие пользователя и ожидаемый результат.

Вот основные сценарии, которые я проверял на десктопе: добавление нового слова через модальное окно (с подбором картинки через Unsplash); редактирование существующего слова; удаление

слова с подтверждением; копирование слова в ту же папку; перемещение слова в другую папку через окно выбора; переворот карточки по клику; живой поиск с фильтрацией секций; запуск режима повторения по одной папке и по всем словам; переключение направления EN → RU / RU → EN; создание пользовательской папки; удаление пользовательской папки; импорт словаря SMARTool A1 по ссылке; импорт собственного CSV-файла; работа кнопок «Знаю» и «Не знаю» в режиме повторения; корректное отображение пустого состояния при удалении всех слов.

На смартфоне акцент был на адаптивности: корректно ли скрывается заголовок на маленьких экранах, умещаются ли кнопки действий в одну строку, не перекрывает ли липкая лента вкладок содержимое, удобно ли нажимать на карточки пальцем (минимальный размер touch-target — 44px по рекомендациям Apple HIG). Оказалось, что на ширине 360px сетка карточек становится слишком тесной — я уменьшил минимальную ширину колонки с 180px до 120px и поправил отступы. Также обнаружилось, что на iOS Safari возникала проблема с `position: sticky` внутри `overflow: auto` — её удалось решить добавлением `-webkit-overflow-scrolling: touch`.

Всего я зарегистрировал и исправил 11 дефектов разной степени серьёзности, из которых 4 — критические. О них подробно в следующем разделе.

3.2. Четыре ошибки, которые меня научили

За время разработки я совершил четыре ошибки, каждая из которых чему-то меня научила. Делюсь ими, потому что они, скорее всего, типовые для начинающего разработчика.

Ошибка 1: переполнение LocalStorage из-за Base64. В первой версии я хотел дать пользователю возможность загружать собственные фотографии. Реализовал это через `FileReader.readAsDataURL`, который превращает бинарный файл в Base64-строку и кладёт её прямо в `imageUri`. Приложение прекрасно работало, пока пользователь не загружал пятую-шестую фотографию — после этого браузер начинал выбрасывать исключение `QuotaExceededError`, и все новые данные про-

сто не сохранялись. Я не сразу понял причину: одна фотография среднего качества в Base64 занимает около 200 КБ, а лимит LocalStorage — всего 5 МБ. Решение: хранить в `imageUrl` только прямые ссылки на Unsplash, а загрузку пользовательских фото отложить до будущей версии (можно будет использовать IndexedDB, у которого лимит измеряется гигабайтами).

Ошибка 2: битые ссылки на картинки и заглушка. Изначально я загружал картинки с сервиса `source.unsplash.com`, который по запросу `https://source.unsplash.com/600x400/?apple` отдаёт случайную фотографию по теме. Этот сервис был удобен тем, что не требует API-ключа. Но в какой-то момент Unsplash закрыл `source.unsplash.com`, и все мои 144 картинки разом превратились в битые иконки. Пришлось срочно заменить их на прямые ссылки вида `https://images.unsplash.com/photo-...`, которые ведут на конкретные фотографии. Я выбрал 144 конкретных фото, проверил каждую вручную и заодно добавил в `cardHTML` обработчик `onerror`, который подменяет битую ссылку на плейсхолдер с текстом «нет картинки». Урок: нельзя зависеть от внешнего сервиса без запасного плана.

Ошибка 3: видимый перевод вместо скрытого. В одной из ранних версий я по неосторожности показывал английский перевод сразу на лицевой стороне карточки, мелким шрифтом под русским словом. Получалось, что мозг пользователя даже не пытается вспомнить перевод — он просто читает его. Эффект активного вспоминания (см. главу 1.1) исчезал полностью. Я убрал перевод с лицевой стороны и оставил только подсказку «нажми — по-английски». С этого момента карточка действительно стала заставлять вспоминать. Урок: маленькое визуальное решение может разрушить всю педагогическую логику приложения.

Ошибка 4: потеря контекста ИИ-ассистентом и появление SPEC.md. Когда я работал с ИИ-ассистентом над ранними версиями проекта, периодически происходила неприятная вещь: я просил поправить одну функцию, а ассистент переписывал половину файла, изменяя имена функций или забывая про какую-то ветку логики. Длинные диалоги приводили к тому, что контекст терялся. Решение пришло не сразу: я стал вести файл `SPEC.md`, в котором по пунктам опи-

сывал все функции, структуру данных, палитру и поведение интерфейса. Перед каждым новым обращением к ассистенту я открывал SPEC.md и точно формулировал задачу. Так появился метод «vibe coding» с обязательной спецификацией — это оказалось быстрее и надёжнее, чем надеяться на память ассистента.

3.3. Инструкция пользователя

Приложение открывается в браузере двойным кликом по файлу `index.html`. Никакой установки не требуется. При первом запуске в LocalStorage копируется встроенный словарь из 144 слов — и вы сразу видите 12 системных папок с карточками.

Просмотр слов. Прокрутите ленту вкладок в верхней панели и выберите нужную папку. В основной области появится секция с карточками этой папки, отсортированными по английскому слову. Чтобы перевернуть карточку и увидеть английский перевод, кликните по ней. Заголовок секции «липнет» к верху при прокрутке, поэтому вы всегда видите, в какой папке находитесь.

Поиск. Введите часть русского или английского слова в поле поиска. Секции, в которых нет совпадений, скроются; останутся только те, где есть хотя бы одно подходящее слово.

Добавление слова. Нажмите «+ Слово» в верхней панели. В модальном окне заполните поля «Английское слово», «Перевод», «Пример», выберите папку. Картинку можно подобрать через Unsplash: введите запрос (например, *apple*), нажмите «🔍 Найти», выберите одно из трёх превью. Либо вставьте прямую ссылку на изображение в поле URL и нажмите «Превью».

Режим повторения. Нажмите «🎓 Повторение» в верхней панели (для всех слов) или «🎓 Повторить группу» в заголовке секции (для одной папки). В полноэкранном оверлее появится большая flip-карточка с картинкой и словом. Кликните по карточке, чтобы увидеть перевод и пример. Отметьте «✓ Знаю» или «✗ Не знаю», затем нажмите «Дальше →». В конце вы увидите статистику: сколько слов пройдено, сколько «знаю», сколько «не знаю».

Импорт словаря. Нажмите « Импорт». Выберите режим раскладки («По уровням и темам», «Только по уровням», «Только по темам», «По алфавиту», «Все в одну папку»). Загрузите файл (.csv / .txt / .json) или вставьте ссылку на CSV SMARTool (например, https://raw.githubusercontent.com/smartool/data-rus-eng/main/SMARTool_data_A1.csv). После завершения импорта вы увидите статус « Импортировано слов: N. Создано папок: M.».

3.4. Опрос 15 одноклассников

Чтобы оценить, насколько приложение действительно помогает учить английский, я провёл опрос среди 15 одноклассников. Каждому я давал приложение на 10 минут и просил ответить на три вопроса. Результаты приведены в Приложении В, здесь — основные выводы.

Вопрос 1. «Легче ли тебе вспоминать слова, когда к ним приложена картинка?» Из 15 опрошенных 13 ответили «да», 2 — «не заметил разницы». Это подтверждает первый принцип памяти — двойное кодирование Пайвио: визуальный образ действительно помогает мозгу быстрее находить нужное слово. Те двое, кто не заметил разницы, вероятно, относятся к аудиальному типу learners — им, возможно, лучше подошло бы произношение через Web Speech API (об этом — в заключении).

Вопрос 2. «Захочешь ли ты заходить в режим повторения каждый день?» 11 ответили «да», 4 — «возможно». Никто не ответил «нет». Это означает, что приложение вызывает желание возвращаться — важнейший показатель для любого обучающего продукта. Один из одноклассников предложил добавить push-уведомления — это, конечно, требует бэкенда и в текущей архитектуре невозможно, но мысль хорошая.

Вопрос 3. «Порекомендуешь ли ты приложение друзьям?» 14 ответили «да», 1 — «не уверен». Это 93% положительных рекомендаций, что для школьного проекта — отличный показатель. Особенно приятно, что несколько одноклассников сразу захотели импортировать собственные словари к подготовке к ОГЭ.

Из открытых комментариев чаще всего звучали две просьбы: добавить озвучку слов и сделать экспорт прогресса в файл. Обе идеи учтены в плане развития (см. заключение).

ЗАКЛЮЧЕНИЕ

Цель проекта — разработать бесплатное веб-приложение для изучения английской лексики методом визуальных ассоциаций, которое объединит тематические папки, карточки-перевёртыши, режим самопроверки и импорт реальных словарей, — достигнута. Все шесть задач, поставленных во введении, выполнены: изучены теоретические основы памяти, проведён обзор аналогов, спроектирована архитектура, реализован встроенный словарь из 144 слов, реализованы карточки-перевёртыши и режим повторения, реализован импорт словарей SMARTool, проведено тестирование и опрос одноклассников.

Главный личный итог — я прошёл полный цикл разработки программного продукта: от анализа предметной области и формулирования требований до тестирования и сбора обратной связи. Это включает в себя не только программирование на HTML, CSS и JavaScript, но и проектирование интерфейса, выбор визуального стиля, написание пояснительной записки. Каждый шаг я документировал, каждое решение обосновывал — что очень отличается от написания кода «в стол» ради сдачи домашнего задания.

Особенно ценным оказался опыт «vibe coding» — совместной работы с ИИ-ассистентом по спецификации SPEC.md. Я научился формулировать технические требования так, чтобы их можно было однозначно проверить, и понял, что ИИ-ассистент не заменяет разработчика, а ускоряет рутинные операции. Без SPEC.md ассистент терял контекст, со SPEC.md — работал точно и предсказуемо. Этот навык, я уверен, пригодится мне и в будущей профессиональной деятельности.

Перспективы развития. В ближайшее время я планирую добавить четыре крупные функции. **Первое** — интеграция с Web Speech API: браузер умеет произносить текст на английском, и озвучка каждого слова сделает приложение полезным для аудиальных learners.

Второе — экспорт словарей в файл JSON или CSV, чтобы пользователь мог перенести свои данные на другой компьютер или поделиться с другом. **Третье** — синхронизация через облачное хранилище (например, через Firebase или собственный мини-сервер), чтобы словарь был доступен с любого устройства. **Четвёртое** и самое амбициозное — автоматическое планирование повторений по системе Лейтнера: приложение само будет предлагать слова на сегодня, основываясь на том, как пользователь отвечал в прошлые дни (метки «знаю» / «не знаю» уже собираются).

Проект доказал мне, что небольшое веб-приложение, написанное школьником на чистом JavaScript, может быть по-настоящему полезным — 14 из 15 одноклассников порекомендовали бы его друзьям. Это лучший мотиватор продолжать.

СПИСОК ИСТОЧНИКОВ

1. Paivio A. Dual coding theory and education. — University of Western Ontario, 1986. — URL: <https://files.eric.ed.gov/fulltext/ED282850.pdf>
2. Флэнаган Д. JavaScript: подробное руководство. — 7-е изд. — СПб.: Символ-Плюс, 2020. — 1088 с.
3. Робсон Э., Фримен Э. Изучаем HTML, XHTML и CSS. — СПб.: Питер, 2017. — 656 с.
4. Duckett J. HTML and CSS: Design and Build Websites. — Wiley, 2011. — 512 p.
5. MDN Web Docs. JavaScript Reference. — URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript>
6. MDN Web Docs. Window.localStorage. — URL: <https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage>
7. Unsplash Developer Documentation. Search Photos. — URL: <https://unsplash.com/documentation#search-photos>
8. Википедия. Интервальные повторения. — URL: https://ru.wikipedia.org/wiki/Интервальные_повторения
9. Википедия. Метод Лейтнера. — URL: https://ru.wikipedia.org/wiki/Метод_Лейтнера
10. ГОСТ 7.32-2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017. — 32 с.

11. Материалы курса «Информатика. 7–9 классы». — М.: БИНОМ. Лаборатория знаний, 2023.
12. SMARTool: Russian-English Vocabulary Dataset (CC-BY 4.0). — URL: <https://github.com/smartool/data-rus-eng>
13. Mueller English-Russian Dictionary. — URL: <https://mueller-dict.sourceforge.net/>
14. Leitner S. So lernt man lernen: Der Weg zum Erfolg. — Freiburg: Herder, 1972. — 192 p.
15. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures: PhD thesis. — University of California, Irvine, 2000. — 162 p.

ПРИЛОЖЕНИЕ А. ПОДПИСИ РИСУНКОВ

Рисунок 1. Схема архитектуры приложения «четыре файла и одно хранилище»: index.html, style.css, script.js, data.js и LocalStorage.

Рисунок 2. Главная страница приложения: закреплённая верхняя панель, липкая лента вкладок-папок, основная область с секциями по папкам.

Рисунок 3. Карточка слова «яблоко» — лицевая сторона (русское слово, подсказка) и обратная сторона (английское слово, пример употребления) до и после переворота.

Рисунок 4. Форма добавления слова с блоком поиска картинки через Unsplash API: три превью с подсветкой выбранного.

Рисунок 5. Полноэкранный режим повторения: большая flip-карточка с картинкой, прогресс «N / M», кнопки «Знаю» / «Не знаю» / «Дальше».

Рисунок 6. Окно импорта словаря: выбор режима раскладки по группам, загрузка файла или импорт по ссылке, список источников SMARTool и Mueller.

ПРИЛОЖЕНИЕ Б. ФРАГМЕНТЫ КОДА

Б.1. Фрагмент объекта слова из data.js

```
{
  id: 'w17',
  word: 'apple',
  translation: 'яблоко',
  example: 'I eat a green apple every morning.',
  folderId: 'f1',
  imageUrl: 'https://images.unsplash.com/photo-1568702846914-96b305d2aaeb?
auto=format&fit=crop&w=600&q=60'
}
```

Поле `id` — уникальный идентификатор слова вида `w101 ... wc12`. Поле `folderId` вида `f1 ... f12` указывает на папку, в которую входит слово (внешний ключ на сущность `folder`). Поле `imageUrl` хранит прямую ссылку на изображение с Unsplash, что позволяет не хранить бинарные данные в LocalStorage (см. главу 1.5).

Б.2. Фрагмент функции searchUnsplash()

```
async function searchUnsplash() {
  const q = $('#unsplashQuery').value.trim();
  if (!q) { toast('Введите запрос', 'error'); return; }
  const box = $('#unsplashResults');
  box.innerHTML = ' Ищем...';
  try {
    let results = [];
    if (UNSPLASH_KEY && UNSPLASH_KEY !== 'YOUR_UNSPASH_ACCESS_KEY') {
      const url = `${UNSPLASH_ENDPOINT}?
query=${encodeURIComponent(q)}&per_page=3&client_id=${UNSPLASH_KEY}`;
      const res = await fetch(url);
      if (!res.ok) throw new Error('Unsplash HTTP ' + res.status);
      const data = await res.json();
      results = (data.results || []).map(r => ({
        url: r.urls.regular,
        thumb: r.urls.thumb
      }));
    } else {
      // Заглушка без ключа: три случайных Unsplash-фото по теме
      results = [1, 2, 3].map(() => ({
        url: `https://source.unsplash.com/600x400/${encodeURIComponent(q)}`,
        thumb: `https://source.unsplash.com/200x150/${encodeURIComponent(q)}`
      }));
    }
    // рендер 3 кликабельных превью...
  } catch (e) {
    box.innerHTML = 'Ошибка: ' + e.message + ' ';
  }
}
```

Функция отправляет GET-запрос на REST API Unsplash, передавая параметр `per_page=3` и API-ключ в качестве `client_id`. Ответ парсится как JSON, из него извлекаются поля `urls.regular` и `urls.thumb` для каждого из трёх результатов. Если ключа нет (плейсхолдер `'YOUR_UNSPASH_ACCESS_KEY'`), функция использует резервный источник `source.unsplash.com`, чтобы интерфейс оставался рабочим даже без регистрации разработчика.

ПРИЛОЖЕНИЕ В. РЕЗУЛЬТАТЫ ОПРОСА

Опрос проводился в марте 2026 года. Выборка — 15 одноклассников 9 класса (8 девочек, 7 мальчиков). Каждому респонденту при-

ложение давалось на 10 минут свободного использования, после чего он отвечал на три вопроса.

№	Вопрос	Да	Нет	Не уверен / возможно
1	Легче ли вспоминать слова, когда к ним приложена картинка?	13	0	2
2	Захочешь ли ты заходить в режим повторения каждый день?	11	0	4
3	Порекомендуешь ли ты приложение друзьям?	14	0	1
Средний показатель «Да», %		87%	0%	13%

Из открытых комментариев чаще всего звучали две просьбы: добавить озвучку слов (7 человек) и сделать экспорт прогресса в файл (5 человек). Эти функции вошли в план развития (см. заключение).